Python Common Errors and How to Fix Them

IndexError: list index out of range

An IndexError is raised when you access a list, tuple or string position that does not exist. Valid indexes run from 0 to len(seq) - 1. The most common cause is an off-by-one mistake in a loop, such as looping up to and including len(seq) instead of stopping before it.

```
nums = [10, 20, 30]
for i in range(len(nums) + 1):   # BUG: i reaches 3
    print(nums[i])               # IndexError at i = 3

# Fix 1: range(len(nums))
# Fix 2 (preferred): iterate directly
for n in nums:
    print(n)
```

Negative indexes count from the end, so nums[-1] is the last item. Accessing nums[-4] on a three item list also raises IndexError. Accessing an empty list with nums[0] raises it too, so check if nums: before indexing.

KeyError

A KeyError is raised when a dictionary key is missing. Use the in operator, the dict.get(key, default) method, or collections.defaultdict to avoid it.

```
ages = {"asha": 21}
print(ages["ravi"])               # KeyError: 'ravi'
print(ages.get("ravi", "unknown"))  # prints unknown
```

TypeError

A TypeError means an operation was applied to a value of the wrong type. Typical examples are adding a string and an int, calling an object that is not callable, passing the wrong number of arguments to a function, and using None where a value was expected.

```
age = 20
print("Age: " + age)            # TypeError: can only concatenate str to str
print("Age: " + str(age))       # fix: convert first
print(f"Age: {age}")            # better: f-string
```

'NoneType' object is not subscriptable, or unsupported operand type(s) for +: 'NoneType' and 'int', means a variable holds None. This usually happens because a function forgot to return a value, or because a method like list.sort() or list.append() was assigned to a variable. These methods change the list in place and return None.

```
nums = [3, 1, 2]
nums = nums.sort()      # BUG: nums is now None
print(nums[0])          # TypeError: 'NoneType' object is not subscriptable

nums = [3, 1, 2]
nums.sort()             # fix: do not reassign
nums = sorted(nums)     # or use sorted(), which returns a new list
```

NameError

A NameError means Python found a name that has not been defined in the current scope. Causes include a typo in a variable name, using a variable before assigning it, forgetting to import a module, and using a loop variable outside the scope where it exists inside a function.

```
print(total)        # NameError: name 'total' is not defined
total = 0           # define it before use

import math         # forgetting this import gives NameError for math
```

AttributeError

An AttributeError is raised when an object does not have the attribute or method you used. Check spelling and capitalisation, check the object type with type(obj), and remember that None has almost no attributes.

```
name = "python"
name.append("!")    # AttributeError: 'str' object has no attribute 'append'
name = name + "!"   # strings are immutable, build a new string
```

IndentationError and SyntaxError

Python uses indentation to mark blocks. Mixing tabs and spaces, or inconsistent indentation, raises IndentationError. A SyntaxError is raised for missing colons after if, for, while, def and class lines, unmatched brackets or quotes, and using = instead of == in a condition.

```
if x == 5        # SyntaxError: missing colon
    print(x)

if x = 5:        # SyntaxError: use == to compare
```

ValueError and ZeroDivisionError

A ValueError means the type is right but the value is not acceptable, for example int('abc'). A ZeroDivisionError is raised when dividing or taking a modulo by zero. Validate input before converting or dividing, or wrap the risky line in try and except.

```
try:
    n = int(input("Enter a number: "))
    print(10 / n)
except ValueError:
    print("Please enter digits only")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

RecursionError and infinite loops

A RecursionError happens when a recursive function never reaches its base case. An infinite while loop happens when the condition never becomes false, usually because the loop variable is not updated inside the loop. Always confirm that every path moves toward the stopping condition.